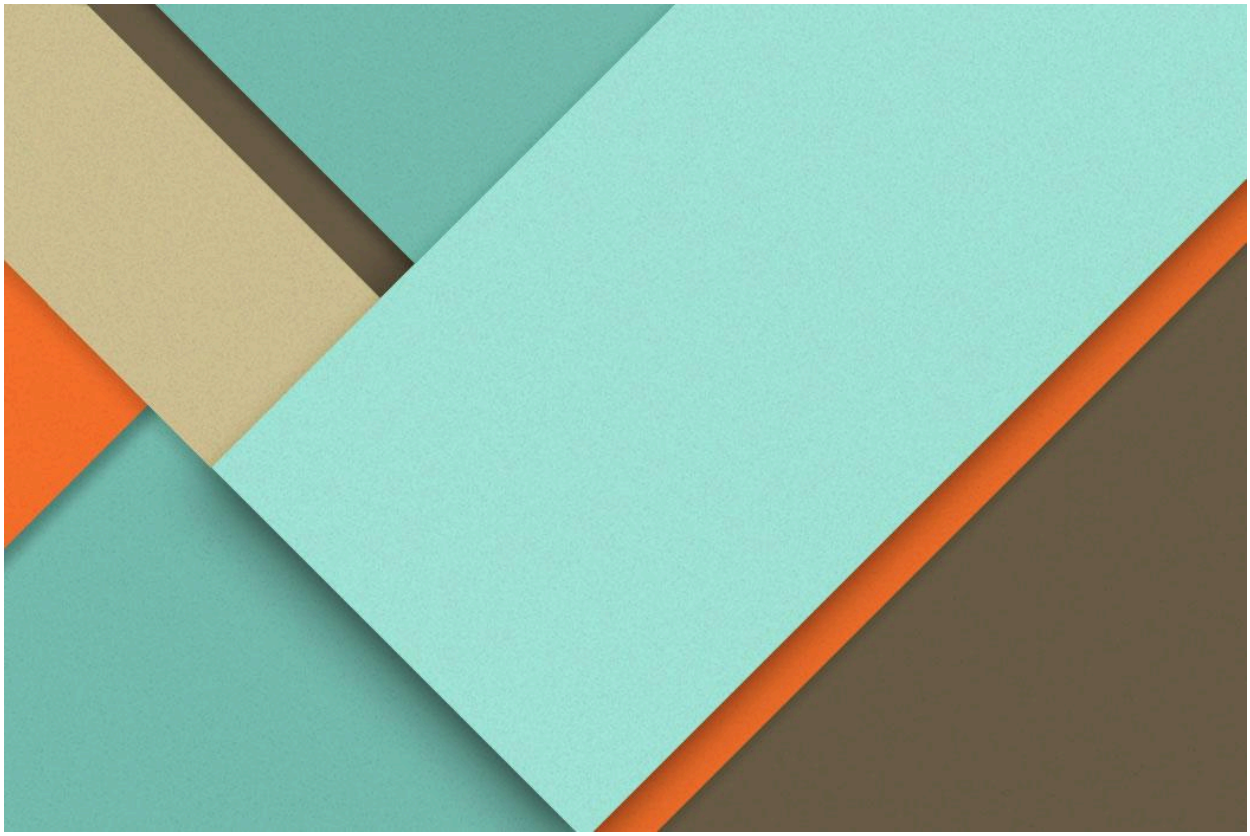

4HACL

07.05.2026

Baptiste CHAPIT
Maxime ARNOULD

Table des matières

1. Introduction
2. Découpage en microservices
3. Infrastructure globale
   - 3.1. Modèle d'écriture et de réplication
   - 3.2. Distribution du compute
   - 3.3. Distribution du contenu — CloudFront
4. Architecture réseau
5. Flux inter-services
   - 5.1. Le parcours de réservation — flux critique
   - 5.2. Les flux secondaires
6. Bases de données et stockage
   - 6.1. Amazon Aurora — données transactionnelles
   - 6.2. DynamoDB — données à haute fréquence
   - 6.3. ElastiCache Redis — cache applicatif et soft lock
   - 6.4. OpenSearch — recherche de vols
   - 6.5. S3 et stratégie d'archivage
   - 6.6. Chiffrement — clés gérées par FlyAway Airlines
7. Sécurité et conformité
   - 7.1. Protection périmétrique
   - 7.2. Chiffrement des données
   - 7.3. Authentification et gestion des secrets
   - 7.4. Détection des menaces et audit
   - 7.5. Conformité RGPD
   - 7.6. Conformité PCI-DSS
8. Observabilité
   - 8.1. Métriques et alerting — CloudWatch
   - 8.2. Tracing distribué — AWS X-Ray
   - 8.3. Logs centralisés — CloudWatch Logs
9. CI/CD
10. Frontend et optimisation mobile
   - 10.1. Distribution du contenu — CloudFront
   - 10.2. Application mobile native
   - 10.3. Cache applicatif — ElastiCache Redis

Introduction

FlyAway Airlines est une compagnie aérienne internationale dont le système d'information existant, reposant sur une architecture microservices, n'a pas répondu aux attentes de performance et de fiabilité. Dans ce contexte, notre équipe a été mandatée pour repenser intégralement l'écosystème technique de la compagnie, de l'infrastructure cloud jusqu'à l'organisation des services applicatifs.

Ce document présente l'architecture cible proposée. La solution doit répondre à des exigences ambitieuses : supporter des dizaines de milliers d'utilisateurs simultanés, garantir une disponibilité mondiale avec une latence minimale, assurer l'intégrité absolue des données transactionnelles, et se conformer aux réglementations RGPD et PCI-DSS. Elle doit également servir deux populations d'utilisateurs aux besoins distincts : les voyageurs, via une interface optimisée mobile, et le personnel de la compagnie, pour la gestion opérationnelle des vols et des passagers.

L'ensemble de l'infrastructure repose sur AWS comme fournisseur cloud unique. Ce choix est motivé par la maturité de son écosystème de services managés, la cohérence des intégrations entre services, et ses certifications de conformité reconnues couvrant PCI-DSS et RGPD. L'ensemble des choix présentés dans ce document — organisation réseau, découpage applicatif, stratégies de stockage et dispositifs de sécurité — sont motivés par ces contraintes et argumentés en fonction des compromis identifiés.

Découpage en microservices

L'architecture applicative repose sur dix services autonomes, chacun responsable de son propre domaine fonctionnel et de ses données. Ce découpage en microservices est retenu car il permet de scaler indépendamment les services selon leur charge respective — le SearchService lors des pics de recherche, le BookingService lors des ouvertures de ventes — sans impacter l'ensemble du système. Il permet également de déployer et mettre à jour chaque service indépendamment, réduisant le risque d'une mise en production.

Service	Responsabilité	Stockage
Cognito	Authentification, sessions, JWT (2 User Pools)	Managé AWS
UserService	Profil métier, préférences, rôles applicatifs	Aurora

DocumentService	Documents sensibles (passeports, pièces d'identité)	S3 + KMS
FlightService	Référentiel vols, sièges, disponibilité temps réel	Aurora
SearchService	Indexation et recherche de vols, géolocalisation aéroports	OpenSearch
BookingService	Orchestration réservation (pattern Saga)	Aurora
PaymentService	Tokenisation, délégation à Stripe/Adyen (PCI-DSS isolé)	Aurora
NotificationService	Emails, SMS, confirmations (mode asynchrone uniquement)	SQS/SNS
HistoryService	Historique réservations et factures (stockage étagé)	DynamoDB + S3 Glacier
StaffService	Validation voyageurs, gestion billets, usage interne	Aurora

Trois décisions méritent d'être soulignées. Premièrement, l'authentification est entièrement déléguée à Amazon Cognito, service managé par AWS qui prend en charge nativement la création de comptes, la gestion des sessions, le MFA et la délivrance de tokens JWT — autant de fonctionnalités qu'il serait coûteux et risqué de développer et maintenir soi-même. Le UserService ne gère que les données métier complémentaires que Cognito ne couvre pas. Deuxièmement, le PaymentService est isolé dans un périmètre réseau dédié et ne stocke jamais de données bancaires en clair, déléguant le traitement à un prestataire certifié PCI-DSS de niveau 1, ce qui réduit drastiquement la portée de la conformité PCI-DSS pour FlyAway Airlines. Troisièmement, l'HistoryService applique une stratégie de stockage étagé : données des douze derniers mois dans DynamoDB pour un accès immédiat, au-delà dans S3 Glacier Instant Retrieval pour réduire les coûts sans impact sur l'UX, et au-delà de sept ans dans S3 Glacier Deep Archive pour répondre aux seules obligations légales de conservation.

Infrastructure globale

L'infrastructure de FlyAway Airlines est déployée sur trois régions AWS : eu-west-1 (Irlande) comme région primaire, us-east-1 (Virginie) pour les Amériques, et ap-southeast-1 (Singapour) pour l'Asie-Pacifique. Ce déploiement multi-région garantit une faible latence

pour les utilisateurs internationaux et une haute disponibilité en cas de défaillance régionale.

Modèle d'écriture et de réplication

Le modèle retenu est primary-write / replica-read : toutes les écritures transitent vers eu-west-1, tandis que les lectures sont servies localement par la région la plus proche de l'utilisateur. Ce choix privilégie la simplicité opérationnelle et l'intégrité des données sur une architecture active-active en écriture, dont la gestion des conflits serait incompatible avec les exigences de cohérence forte du système (réservations, paiements).

Aurora Global Database assure la réplication des données transactionnelles entre les trois régions avec un lag typique inférieur à une seconde. En cas de défaillance d'eu-west-1, Route 53 détecte la panne via ses health checks et Aurora promeut automatiquement us-east-1 en région primaire en moins d'une minute, avec une perte de données quasi nulle.

DynamoDB Global Tables est utilisé pour les données à haute fréquence et sans relation complexe (sessions, historique récent, préférences). Son modèle multi-région natif en écriture est acceptable ici car ces données ne sont jamais modifiées simultanément depuis deux régions, rendant les conflits quasi impossibles.

OpenSearch est répliqué dans les trois régions en lecture seule, alimenté par le FlightService depuis eu-west-1. La recherche de vols étant une opération de lecture pure, cette réplication améliore directement la latence pour les utilisateurs américains et asiatiques.

Distribution du compute

Les services applicatifs sont déployés sur Amazon EKS (Kubernetes managé) dans les trois régions. EKS est retenu car il permet de scaler automatiquement le nombre de pods en fonction de la charge (via l'autoscaler horizontal), de distribuer les instances sur plusieurs Availability Zones sans configuration manuelle, et de déléguer la gestion du plan de contrôle Kubernetes à AWS. Cette dernière caractéristique est particulièrement importante pour réduire la charge opérationnelle sur une infrastructure déployée dans trois régions simultanément.

Le PaymentService est confiné à eu-west-1 uniquement. Ce choix réduit le périmètre d'audit PCI-DSS à une seule région, simplifiant considérablement la conformité. Les utilisateurs des autres régions effectuent leurs paiements via un appel cross-région vers eu-west-1 (ce qui est acceptable car la réservation est une action ponctuelle et non du temps réel).

Chaque cluster EKS est déployé sur trois Availability Zones distinctes au sein de sa région, garantissant qu'une défaillance matérielle ou réseau sur une zone n'entraîne aucune interruption de service.

Distribution du contenu — CloudFront

CloudFront est déployé en frontal de l'ensemble de l'infrastructure. Il met en cache les assets statiques (interfaces, images, fichiers JavaScript et CSS) dans plus de 400 points de présence mondiaux, réduisant la latence pour tous les utilisateurs indépendamment de leur région. CloudFront constitue également le point d'entrée du WAF, permettant de filtrer les attaques avant qu'elles n'atteignent l'infrastructure. Le routage entre CloudFront et les régions AWS est assuré par Route 53 en mode latency-based, qui dirige chaque utilisateur vers la région AWS offrant la latence la plus faible à l'instant de la requête, avec bascule automatique vers une région secondaire en cas de défaillance.

Architecture réseau

L'architecture réseau repose sur un VPC dédié par région AWS, structuré en trois couches de sous-réseaux isolés. Ce découpage en couches est retenu pour minimiser la surface d'attaque : chaque couche n'est accessible que depuis celle qui la précède, de sorte qu'une compromission de la couche publique ne donne pas accès directement aux bases de données. La couche publique accueille les Application Load Balancers et les NAT Gateways. La couche privée héberge les pods EKS et les fonctions Lambda. La couche isolée, sans aucune route vers internet, est réservée aux bases de données (Aurora, DynamoDB, ElastiCache, OpenSearch) et n'est accessible que depuis la couche privée. Chaque couche est répliquée sur trois Availability Zones distinctes au sein de chaque région, garantissant qu'une défaillance matérielle sur une zone n'entraîne aucune interruption de service.

Le flux entrant suit le chemin suivant : Route 53 oriente chaque requête vers la région la plus proche, CloudFront sert les assets statiques depuis ses points de présence mondiaux, puis le trafic est transmis à l'Application Load Balancer en couche publique, qui le distribue vers les pods EKS en couche privée. Le trafic sortant des microservices vers internet transite obligatoirement par une NAT Gateway (permettant aux services d'initier des connexions sortantes sans disposer d'adresse IP publique directe, éliminant ainsi tout risque d'accès entrant non sollicité).

La connectivité entre les trois régions est assurée par AWS Transit Gateway avec inter-region peering, garantissant que le trafic inter-région transite exclusivement sur le backbone privé d'AWS sans jamais passer par internet. Ce choix est important pour la conformité : les données répliquées entre régions (réservations, profils, documents) ne

transitent jamais sur un réseau public. Les communications entre les microservices et les services managés AWS (S3, DynamoDB, SQS, etc.) s'effectuent via des VPC Endpoints (PrivateLink), assurant que ce trafic ne quitte jamais le réseau privé AWS (un prérequis explicite pour la conformité RGPD et PCI-DSS).

Enfin, des Security Groups sont appliqués à chaque microservice, définissant précisément quels services sont autorisés à communiquer entre eux. Un service n'accepte que le trafic provenant des services qui ont explicitement besoin de lui parler, limitant la surface d'attaque en cas de compromission.

Flux inter-services

L'architecture distingue deux modes de communication entre services : les appels REST synchrones pour les opérations critiques nécessitant une réponse immédiate, et la messagerie asynchrone via SNS/SQS pour les opérations secondaires découplées de l'expérience utilisateur.

Le parcours de réservation — flux critique

Dès qu'un utilisateur sélectionne un siège, le FlightService enregistre un soft lock dans ElastiCache Redis avec un TTL de 10 minutes. Redis est retenu pour cette fonction car il gère nativement l'expiration automatique des clés (évitant d'implémenter un mécanisme de nettoyage et d'utiliser Aurora pour des données temporaires à courte durée de vie). Le siège est automatiquement libéré à l'issue du délai si la réservation n'est pas finalisée.

L'orchestration est assurée par le BookingService via des appels REST synchrones. Ce choix est délibéré : sur une opération financière, l'utilisateur a besoin d'une réponse immédiate et déterministe. Une architecture basée sur des queues de messages rendrait la gestion des erreurs significativement plus complexe et l'expérience utilisateur dégradée en cas d'échec. Le flux se déroule comme suit :

1. Le BookingService vérifie la disponibilité auprès du FlightService
2. Il initie le paiement auprès du PaymentService, avec un timeout de 30 secondes. Au-delà, le siège est libéré et l'utilisateur invité à réessayer
3. En cas de succès, la réservation est confirmée en base Aurora
4. En cas d'échec à n'importe quelle étape, le BookingService déclenche les compensations nécessaires : libération du siège et remboursement si le paiement avait été initié

La concurrence est gérée par un verrou optimiste en base : chaque siège dispose d'un numéro de version vérifié au moment de l'écriture. Ce mécanisme est préféré au verrou pessimiste (qui bloquerait séquentiellement tous les accès concurrents et créerait un

goulot d'étranglement inacceptable à l'échelle de dizaines de milliers d'utilisateurs simultanés).

Une fois la réservation confirmée, le BookingService publie un événement sur Amazon SNS. SNS est retenu ici car il permet de diffuser un même événement vers plusieurs consommateurs simultanément (NotificationService et HistoryService) sans que le BookingService n'ait à connaître ses destinataires. Chaque consommateur dispose de sa propre file SQS, garantissant qu'un message non traité est conservé jusqu'à sa consommation effective. Ces opérations ne conditionnant pas la réussite de la réservation, leur découplage garantit qu'une défaillance de l'un de ces services n'affecte jamais le processus de réservation.

Les flux secondaires

Les flux secondaires sont significativement plus simples. La recherche de vol transite directement vers le SearchService qui interroge l'index OpenSearch local à la région de l'utilisateur, sans jamais toucher Aurora. La création de compte est gérée nativement par Cognito, qui déclenche la création du profil métier dans le UserService. L'upload de documents sensibles utilise des URLs présignées S3 (des liens temporaires à durée de vie limitée générés par le DocumentService) permettant un upload direct depuis le client vers S3 sans que les fichiers ne transitent par les microservices, réduisant la charge sur l'infrastructure applicative.

Bases de données et stockage

La stratégie de stockage de FlyAway Airlines est différenciée selon la nature des données, leur fréquence d'accès et leurs exigences de cohérence. Utiliser une technologie unique pour l'ensemble des besoins aurait conduit à des compromis inacceptables sur la performance ou la cohérence.

Amazon Aurora — données transactionnelles

Aurora est retenu pour toutes les données nécessitant des garanties ACID (atomicité, cohérence, isolation, durabilité) : réservations, paiements, profils utilisateurs et référentiel de vols. Ces garanties assurent qu'une opération complexe comme une réservation réussit ou échoue dans sa totalité, sans jamais laisser les données dans un état intermédiaire incohérent. Son modèle relationnel permet de modéliser les relations complexes entre ces entités. Aurora Global Database assure la réplication vers us-east-1 et ap-southeast-1 avec un lag inférieur à une seconde, toutes les écritures transitant vers la région primaire eu-west-1.

Les backups automatiques sont configurés avec une rétention de 35 jours (couvrant les délais légaux de contestation des transactions bancaires). Des snapshots manuels mensuels sont conservés 7 ans pour répondre aux obligations légales de conservation comptable.

DynamoDB — données à haute fréquence

DynamoDB est retenu pour les sessions utilisateurs, les préférences et l'historique récent (moins de 12 mois). Ces données ont en commun d'être accessibles par clé simple, sans relation complexe avec d'autres entités, et à très haute fréquence. DynamoDB offre des temps de réponse inférieurs à 10 millisecondes quelle que soit la charge, et scale automatiquement sans intervention manuelle (deux propriétés qu'Aurora ne peut pas garantir à ce niveau). DynamoDB Global Tables assure la réplication multi-région native.

ElastiCache Redis — cache applicatif et soft lock

Redis remplit deux fonctions dans l'architecture. Premièrement, le soft lock de réservation décrit dans la section précédente, exploitant le TTL natif de Redis pour la libération automatique des sièges. Deuxièmement, un cache applicatif devant Aurora pour les données fréquemment lues et rarement modifiées (données de profil, référentiel de vols), réduisant significativement la charge sur Aurora et améliorant les temps de réponse. Redis est préféré à une solution de cache personnalisée car il est managé, hautement disponible, et intégré nativement à l'écosystème AWS.

OpenSearch — recherche de vols

OpenSearch est un moteur d'indexation et de recherche optimisé pour les requêtes complexes et multi-critères, là où Aurora serait trop lent et trop rigide. Il permet des recherches combinant destination, plage de dates, classe, prix et géolocalisation des aéroports en quelques millisecondes. L'index est alimenté par le FlightService depuis eu-west-1 et répliqué en lecture dans les deux autres régions, garantissant une latence minimale pour les utilisateurs internationaux.

S3 et stratégie d'archivage

Amazon S3 est retenu pour le stockage des documents sensibles (passeports, pièces d'identité) et l'archivage de l'historique. S3 est choisi pour sa durabilité garantie par AWS (99,999999999%), sa capacité à stocker des fichiers de taille variable sans contrainte de volume, et son intégration native avec KMS pour le chiffrement et les URLs présignées pour l'accès sécurisé. L'historique suit une stratégie de stockage étagé : S3 Glacier Instant Retrieval au-delà de 12 mois (récupération immédiate, coût réduit de 70% par rapport à S3

Standard), et S3 Glacier Deep Archive au-delà de 7 ans (conservation à coût minimal pour répondre aux seules obligations légales).

Chiffrement – clés gérées par FlyAway Airlines

L'ensemble des données au repos est chiffré via AWS KMS avec des clés gérées par FlyAway Airlines (Customer Managed Keys). Le choix de gérer ses propres clés plutôt que de déléguer à AWS est motivé par trois raisons. Premièrement, les obligations PCI-DSS et RGPD imposent à FlyAway Airlines de prouver qu'elle seule contrôle l'accès à ses données sensibles (ce que les clés AWS managées ne permettent pas, AWS conservant techniquement un accès). Deuxièmement, des clés distinctes sont définies par domaine fonctionnel (paiements, documents personnels, données générales), permettant une révocation granulaire en cas d'incident de sécurité sans impacter l'ensemble du système. Troisièmement, la souveraineté des données est un enjeu réel pour une compagnie traitant des données de passeports et des données bancaires internationales.

Sécurité et conformité

La sécurité de l'architecture FlyAway Airlines repose sur une approche en profondeur : chaque couche dispose de ses propres mécanismes de protection, de sorte qu'une compromission isolée ne suffit pas à exposer les données sensibles.

Protection périmétrique

AWS WAF est déployé devant CloudFront et les Application Load Balancers. Il filtre les requêtes malveillantes avant qu'elles n'atteignent l'infrastructure : injections SQL, cross-site scripting, exploitation des failles OWASP Top 10, et limitation du nombre de requêtes par IP pour contrer les abus. WAF est retenu car ses règles managées par AWS sont mises à jour en continu face aux nouvelles menaces, sans nécessiter de composant tiers.

AWS Shield Advanced assure la protection contre les attaques DDoS volumétriques. Contrairement à Shield Standard inclus gratuitement, Shield Advanced offre une mitigation automatique des attaques sophistiquées et un accès à l'équipe de réponse DDoS d'AWS. Pour une compagnie aérienne dont la disponibilité est critique, une attaque non mitigée pendant une période de forte demande aurait un impact business direct.

Chiffrement des données

Toutes les communications externes sont chiffrées via TLS 1.3, le protocole standard qui garantit que les données échangées entre le client et les serveurs ne peuvent pas être

interceptées ou lues par un tiers. Les communications internes entre microservices utilisent le mTLS (mutual TLS), une variante où chaque service vérifie l'identité de son interlocuteur avant d'accepter une connexion — empêchant qu'un service compromis puisse se faire passer pour un autre. Cette couche de sécurité est gérée par AWS App Mesh, un service mesh qui intercepte et sécurise automatiquement toutes les communications entre pods EKS sans modifier le code applicatif.

Le chiffrement des données au repos est appliqué sans exception à Aurora, DynamoDB, ElastiCache, OpenSearch et S3 via AWS KMS avec des clés gérées par FlyAway Airlines, détaillé dans la section bases de données.

Authentification et gestion des secrets

Le MFA (authentification à double facteur) est imposé pour l'ensemble du personnel de la compagnie, qui dispose de droits d'accès étendus sur les données passagers. Pour les voyageurs, il est disponible mais optionnel afin de ne pas dégrader l'expérience utilisateur. Aucune clé d'accès statique n'est utilisée dans l'infrastructure — les microservices s'authentifient via IRSA, les pipelines CI/CD via des rôles IAM à permissions limitées.

Les credentials (mots de passe de bases de données, clés API Stripe et Adyen) sont centralisés dans AWS Secrets Manager avec rotation automatique périodique. Cet outil élimine les credentials statiques en dur dans le code et permet de révoquer un secret compromis en quelques secondes depuis un point de contrôle unique.

Détection des menaces et audit

AWS GuardDuty analyse en continu les logs réseau et les comportements des services pour détecter les anomalies : tentatives d'accès inhabituelles, communications vers des adresses malveillantes, comportements anormaux des credentials IAM. AWS Macie surveille spécifiquement les buckets S3 pour détecter toute exposition accidentelle de données personnelles — particulièrement pertinent pour un système stockant des copies de passeports. AWS Security Hub agrège les alertes de ces deux services en une vue unifiée multi-région. L'ensemble des actions sur l'infrastructure est audité via CloudTrail, dont les logs immuables sont conservés dans un bucket S3 dédié.

Conformité RGPD

Les données personnelles des utilisateurs européens sont confinées à eu-west-1 via une politique de geofencing. Le droit à l'effacement est implémenté par pseudonymisation : les données identifiantes (nom, email, téléphone) sont remplacées par un identifiant anonyme, tandis que les données transactionnelles sont conservées pour répondre aux obligations légales de 7 ans. Cette approche est explicitement reconnue par le RGPD

comme méthode valide. Les documents sensibles stockés dans S3 sont quant à eux supprimés physiquement, n'ayant aucune obligation de conservation.

Conformité PCI-DSS

Le PaymentService est isolé dans un périmètre réseau dédié et confiné à eu-west-1, limitant la surface d'audit PCI-DSS à ce seul service. FlyAway Airlines ne manipule aucune donnée bancaire en clair, déléguant intégralement le traitement des cartes à un prestataire certifié PCI-DSS de niveau 1 (Stripe ou Adyen) et ne conservant que les tokens de paiement retournés par ce prestataire.

Observabilité

Dans une architecture microservices distribuée sur trois régions, la capacité à détecter, localiser et diagnostiquer rapidement un incident est aussi critique que la fiabilité de l'infrastructure elle-même. L'observabilité repose sur trois piliers complémentaires, tous intégrés nativement à l'écosystème AWS.

Métriques et alerting – CloudWatch

AWS CloudWatch collecte en continu les métriques de performance de l'ensemble de l'infrastructure : utilisation CPU et mémoire des pods EKS, latence et taux d'erreur des APIs, performance des bases de données Aurora et DynamoDB, profondeur des files SQS. Son intégration native avec l'ensemble des services AWS managés évite de déployer et maintenir un agent de monitoring tiers sur chaque composant.

Des alarmes sont configurées sur les indicateurs critiques — latence anormale sur le BookingService, taux d'erreur élevé sur le PaymentService, réplication Aurora dégradée entre régions — et déclenchent des notifications automatiques vers l'équipe opérationnelle. Des dashboards CloudWatch centralisés offrent une vue temps réel de la santé du système par région : disponibilité des services, volume de réservations, latence globale et état de la réplication multi-région.

Tracing distribué – AWS X-Ray

X-Ray est le composant le plus structurant de l'observabilité dans cette architecture. Lorsqu'une requête traverse successivement le BookingService, le FlightService et le PaymentService, X-Ray trace son chemin complet et mesure précisément le temps passé dans chaque service et chaque appel de base de données. Sans ce niveau de visibilité, identifier la source d'une dégradation de performance dans un système distribué serait extrêmement complexe. X-Ray est retenu car il s'intègre nativement à EKS, API Gateway et

Lambda, et corrèle automatiquement les traces avec les métriques CloudWatch pour une vue unifiée lors d'un incident.

Logs centralisés – CloudWatch Logs

Chaque microservice déployé sur EKS publie ses logs dans CloudWatch Logs. Cette centralisation est indispensable dans une architecture distribuée : lors d'un incident, elle permet de corréler les logs de plusieurs services simultanément sans avoir à se connecter individuellement à chaque pod. Les logs sont conservés avec une rétention différenciée selon leur nature — 30 jours pour les logs applicatifs courants, 1 an pour les logs liés aux opérations de paiement et de réservation.

CI/CD

Le pipeline de livraison repose sur GitLab auto-hébergé, retenu pour maintenir la souveraineté du code source dans l'infrastructure de FlyAway Airlines, en cohérence avec les contraintes RGPD du projet.

À chaque modification du code, GitLab CI déclenche automatiquement un pipeline qui compile le code, exécute les tests automatiques, construit une image Docker du microservice concerné et la pousse dans Amazon ECR, le registre d'images privé d'AWS. Cette étape garantit que seul du code validé et testé peut atteindre l'environnement de production.

Le déploiement sur EKS est ensuite pris en charge par ArgoCD, qui surveille en permanence ECR et applique automatiquement les nouvelles versions sur les clusters des trois régions. Les mises à jour sont effectuées en rolling update : les nouveaux pods sont démarrés progressivement avant que les anciens ne soient arrêtés, garantissant une disponibilité continue du service sans interruption. En cas d'anomalie détectée sur la nouvelle version, ArgoCD effectue un rollback automatique vers la version précédente.

Les pipelines GitLab disposent de rôles IAM dédiés aux permissions strictement limitées aux opérations de déploiement, sans accès possible aux données de production.

Frontend et optimisation mobile

FlyAway Airlines dispose de deux interfaces distinctes : une application web pour les accès desktop, et une application mobile native iOS et Android pour les voyageurs. Le personnel de la compagnie accède au StaffService via une interface web dédiée, séparée de l'interface voyageur.

Distribution du contenu – CloudFront

L'ensemble des assets statiques (JavaScript, CSS, images) est distribué via CloudFront depuis ses 400+ points de présence mondiaux. Quel que soit la localisation de l'utilisateur, ces ressources sont servies depuis le point de présence le plus proche plutôt que depuis eu-west-1, réduisant significativement les temps de chargement initiaux.

Application mobile native

Une application native iOS et Android est retenue plutôt qu'une application web responsive. Ce choix est justifié par la taille de FlyAway Airlines et les attentes de ses utilisateurs : les grandes compagnies aériennes proposent systématiquement une application native, qui offre une expérience significativement supérieure sur mobile.

L'application native exploite plusieurs fonctionnalités du terminal qui ne sont pas accessibles depuis un navigateur web. L'authentification biométrique (Touch ID, Face ID) est supportée nativement via Cognito, offrant une connexion rapide et sécurisée sans saisie de mot de passe. Le scan de passeport utilise directement l'appareil photo du téléphone pour alimenter le DocumentService lors de l'inscription, évitant un upload manuel fastidieux. Enfin, les notifications push sont envoyées directement sur le téléphone via Amazon SNS, qui gère nativement la distribution vers APNS (Apple) et FCM (Google) : rappels de vol, changements de porte d'embarquement, confirmations de réservation en temps réel.

Cache applicatif – ElastiCache Redis

Les données fréquemment consultées et rarement modifiées — informations de vols, données d'aéroports, profil utilisateur — sont mises en cache dans ElastiCache Redis en amont d'Aurora. Ce cache est particulièrement bénéfique sur mobile où la latence perçue est plus sensible qu'en desktop. Il réduit également la charge sur Aurora lors des pics de trafic, comme les périodes d'ouverture des ventes ou les vacances scolaires.